3.10 Residual networks (ResNet)

- as we design increasingly deeper networks, it becomes imperative to understand how adding layers can increase the complexity and expressiveness of the network
- even more important is the ability to design networks where adding layers makes networks strictly more expressive, rather than just different
- this is the question that He et al. considered when working on very deep computer vision models
- at the heart of their proposed *residual network* (*ResNet*) is the idea that every additional layer should more easily contain the *identity function* as one of its elements
- these considerations are rather profound, but they led to a surprisingly simple solution, the *residual block*
- with it, ResNet won the ImageNet Large Scale Visual Recognition Challenge in 2015
- the design had a profound influence on how to build deep neural networks

3.10 Residual networks (ResNet)

- let us focus on a local part of a neural network, as depicted in Figure 14
- denote the input by $\mathbf{x}$, and assume that the desired underlying mapping we want to obtain by learning is $f(\mathbf{x})$, to be used as the input to the activation function on the top
- on the left of Figure 14, the portion within the dotted-line box must directly learn the mapping $f(\mathbf{x})$
- on the right, the portion within the dotted-line box needs to learn the *residual mapping* $f(\mathbf{x}) - \mathbf{x}$, which is how the residual block derives its name
- if the identity mapping $f(\mathbf{x}) = \mathbf{x}$ is the desired underlying mapping, the residual mapping is easier to learn: we only need to push the weights and biases of the upper weight layer (e.g., fully-connected layer and convolutional layer) within the dotted-line box to zero
- the right figure in Figure 14 illustrates the *residual block* of ResNet, where the solid line carrying the layer input $\mathbf{x}$ to the addition operator is called a *residual connection* (or *shortcut connection*)
- with residual blocks, inputs can forward propagate faster through the residual connections across layers

3.10 Residual networks (ResNet)

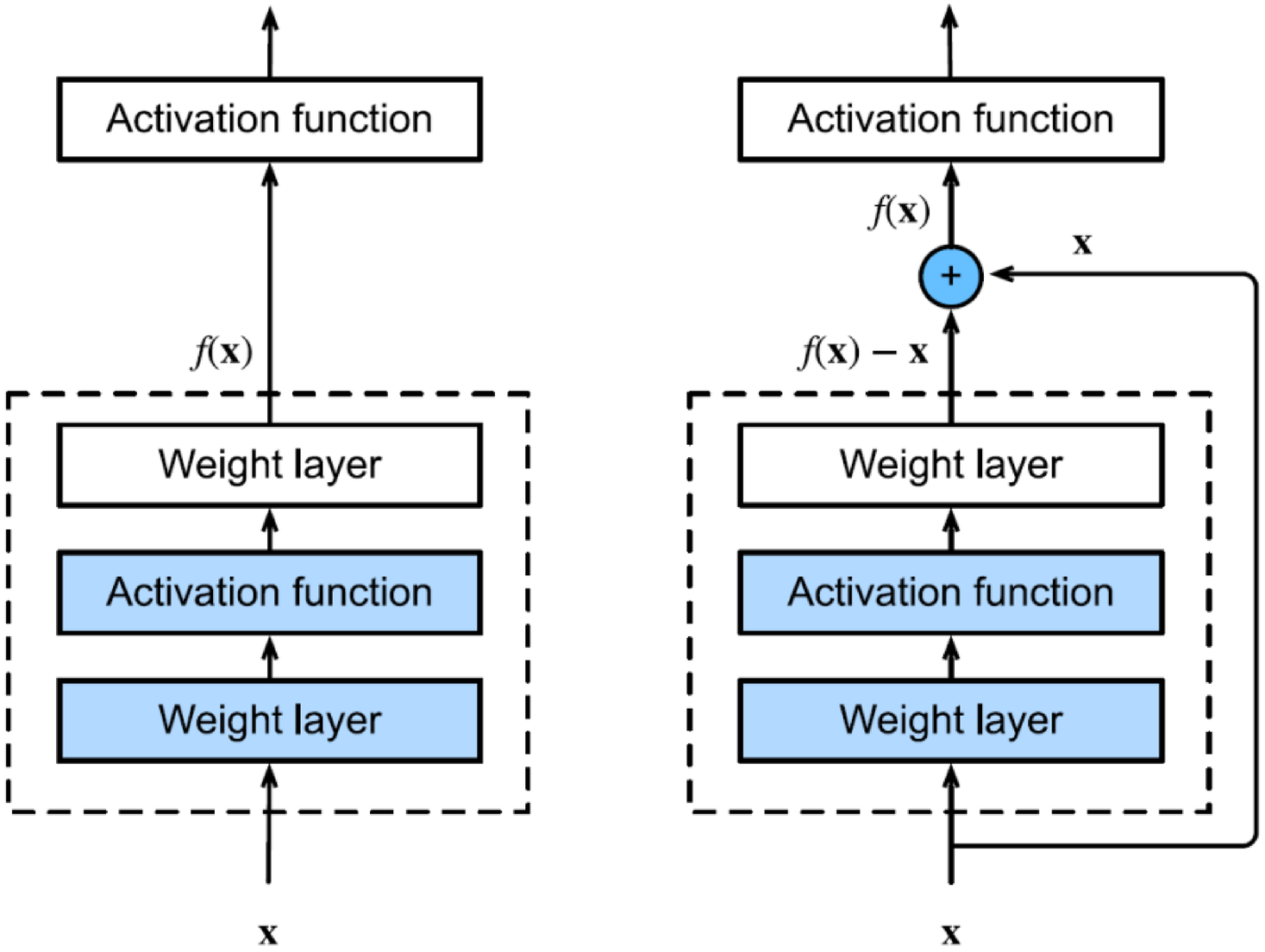

Figure 14: A regular block (left) and a residual block (right).

3.10 Residual networks (ResNet)

- ResNet follows VGG's full 3×3 convolutional layer design
- the residual block has two 3×3 convolutional layers with the same number of output channels
- each convolutional layer is followed by a batch normalization layer and a ReLU activation function
- then, we skip these two convolution operations and add the input directly before the final ReLU activation function
- this kind of design requires that the output of the two convolutional layers has to be of the same shape as the input, so that they can be added together
- if we want to change the number of channels or the resolution, we need to introduce an additional 1×1 convolutional layer to transform the input into the desired shape for the addition operation
- thus, we have two types of blocks: one where we add the input to the output before applying the ReLU nonlinearity, and one where we adjust channels and resolution by means of a 1×1 convolution before adding; Figure 15 illustrates this

3.10 Residual networks (ResNet)

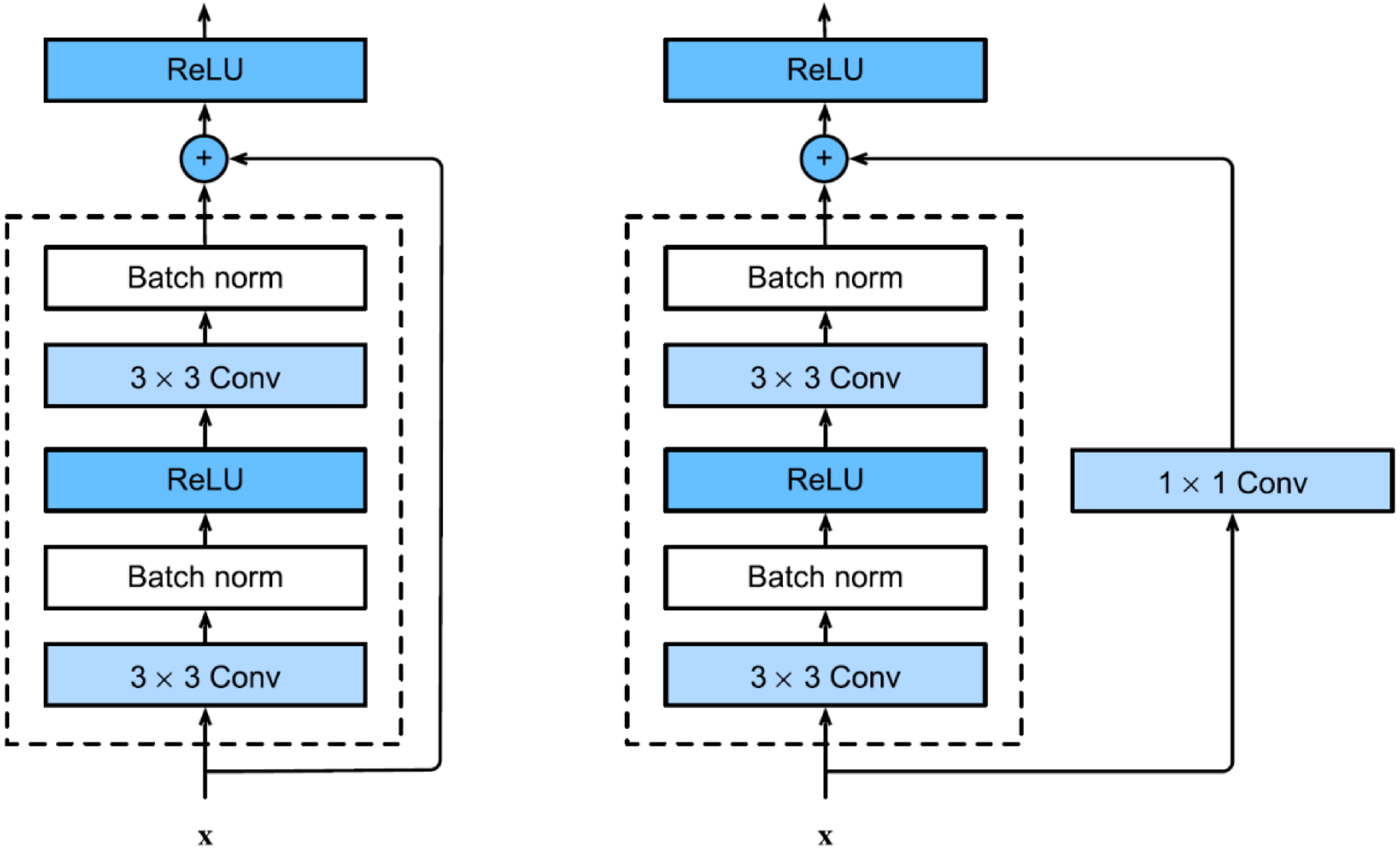

Figure 15: ResNet block with and without 1×1 convolution.

3.10 Residual networks (ResNet)

- the first two layers of ResNet are a 7×7 convolutional layer with 64 output channels and a stride of 2, which is followed by the 3×3 maximum pooling layer with a stride of 2
- a batch normalization layer is added after each convolutional layer in ResNet
- ResNet uses four modules made up of residual blocks, each of which uses several residual blocks with the same number of output channels
- the number of channels in the first module is the same as the number of input channels
- since a maximum pooling layer with a stride of 2 has already been used, it is not necessary to reduce the height and width
- in the first residual block, for each of the subsequent modules, the number of channels is doubled compared with that of the previous module, and the height and width are halved

3.10 Residual networks (ResNet)

- finally, we add a global average pooling layer, followed by the fully-connected layer output
- there are 4 convolutional layers in each module (excluding the 1×1 convolutional layer)
- together with the first 7×7 convolutional layer and the final fully-connected layer, there are 18 layers in total
- therefore, this model is commonly known as *ResNet-18*
- by configuring different numbers of channels and residual blocks in the module, we can create different ResNet models, such as the deeper 152-layer *ResNet-152*
- the main architecture of ResNet is simple and easy to modify, which has resulted in the rapid and widespread use of ResNet; Figure 16 depicts the full ResNet-18

3.10 Residual networks (ResNet)

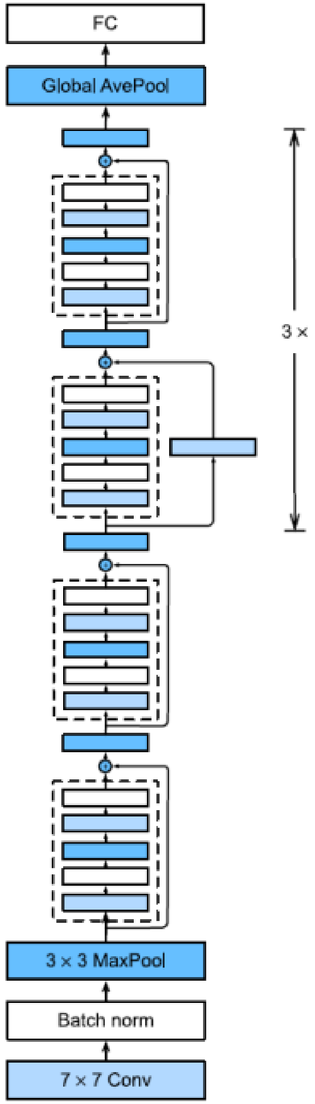

Figure 16: The ResNet-18 architecture.